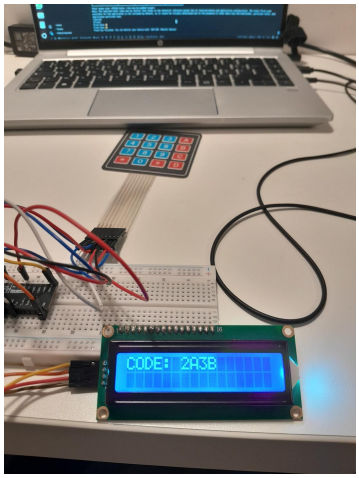
Rapport d'avancement

Système de sécurité intelligent
par code d'accès

Communication ESP32 ↔ Raspberry Pi
via Bluetooth Low Energy (BLE)

Technologies utilisées

ESP32-WROVER-CAM • ESP-IDF 6.1 • FreeRTOS
Bus I2C • LCD 16×2 (PCF8574) • Keypad 4×4
Bluetooth Low Energy (NimBLE) • Raspberry Pi Zero W



Le code « 2A3B » saisi au keypad et affiché en temps réel sur le LCD

Kelly Yann Lionnel Kassin

9 octobre 2026

Résumé

Ce rapport fait le point sur le projet **Système de sécurité intelligent** : un contrôle d'accès par code destiné à un immeuble. Un **ESP32-WROVER-CAM** reçoit la saisie d'un code sur un keypad 4×4, l'affiche en temps réel sur un LCD 16×2 piloté en I2C, puis le transmet en **Bluetooth Low Energy** à un **Raspberry Pi** chargé de le vérifier dans une base de données et de renvoyer une décision (accès accordé ou refusé).

À ce jour, la partie embarquée locale est opérationnelle : bus I2C et pilote LCD écrits au niveau registre avec l'API `i2c_master` d'ESP-IDF 6.1, balayage matriciel du keypad dans une tâche FreeRTOS, serveur GATT NimBLE, et échanges validés avec un smartphone (application nRF Connect). Restent à réaliser la protection des accès concurrents (mutex), le client BLE et la base de données sur le Raspberry Pi, le protocole de décision et la commande de la barrière.

Avancement global estimé : 52,5 %



52,5 %

Mots-clés : ESP32, ESP-IDF, I2C, PCF8574, HD44780, FreeRTOS, BLE, GATT, NimBLE, Raspberry Pi, contrôle d'accès.

Table des matières

Résumé **1**

1 Introduction **4**
1.1 Contexte et objectif . . . 4
1.2 Cahier des charges . . . 4
1.3 Architecture globale . . . 4
1.4 Matériel et environnement logiciel . . . 5

2 État d'avancement **5**

3 Travaux réalisés **6**
3.1 Environnement de développement ESP-IDF . . . 6
3.2 Câblage et affectation des broches . . . 7
3.3 Protocole I2C . . . 9
3.3.1 Principe . . . 9
3.3.2 Mise en œuvre avec l'API `i2c_master` d'ESP-IDF . . . 10
3.3.3 Balayage du bus . . . 10
3.4 Programmation du LCD . . . 10
3.4.1 Rôle du PCF8574 . . . 10
3.4.2 Envoi d'un octet au LCD . . . 11
3.4.3 Séquence d'initialisation du HD44780 . . . 11
3.5 Keypad matriciel 4×4 . . . 12
3.5.1 Principe du balayage . . . 12
3.5.2 Anti-rebond et tâche FreeRTOS . . . 13
3.6 Pile Bluetooth Low Energy (NimBLE) . . . 13
3.6.1 Concepts GATT . . . 13
3.6.2 Démarrage de la pile . . . 14
3.6.3 Callback d'accès . . . 14
3.7 Gestion des tâches et saisie du code . . . 15
3.8 Communication ESP32 ↔ téléphone . . . 15

4 Difficultés rencontrées et solutions **17**

5 Analyse de sécurité et points d'attention **18**

6 Travaux restants **19**
6.1 Mise en place d'un mutex . . . 19
6.2 Connexion du Raspberry Pi au BLE de l'ESP32 . . . 19
6.3 Base de données des habitants . . . 20
6.4 Communication RPi ↔ ESP32 . . . 20

6.5 Vérification des codes et envoi de commandes 21
6.6 Perspectives 21
6.7 Feuille de route 22

7 Conclusion **22**

A Extraits de code **22**
A.1 Bus I2C et balayage 22
A.2 Pilote du LCD 23
A.3 Keypad 23
A.4 Serveur BLE 24

B Commandes utiles **25**

C Gestion de version **25**

Références **26**

Table des figures

1 Architecture globale. Contour plein vert : réalisé et testé ; orange : en cours ; rouge pointillé : à faire . . . 5
2 Poste de travail . . . 7
3 Brochage de la carte . . . 8
4 Fonctions des broches (source tierce) . . . 8
5 Principe du bus I2C . . . 9
6 Résultat du scan I2C . . . 10
7 Structure de l'octet envoyé au PCF8574 . . . 11
8 Premier montage ESP32 + LCD . . . 12
9 Affectation initiale des broches du keypad . . . 13
10 Vérification du keypad . . . 13
11 Hiérarchie GATT du projet . . . 14
12 Service et caractéristique vus par nRF Connect, après écriture d'une chaîne de test 16
13 Lecture par le téléphone du code « 2A3B » saisi au keypad . . . 16
14 Journal de l'ESP32 lors des tests BLE . . . 16
15 Code saisi sur le LCD . . . 17
16 Séquence cible de vérification d'un code . . . 21
17 Feuille de route du projet . . . 22
18 Configuration GitHub . . . 25

Liste des tableaux

1 Exigences fonctionnelles (F) et non fonctionnelles (NF) . . . 4
2 Matériel utilisé . . . 5
3 Environnement logiciel . . . 5
4 Avancement détaillé par tâche . . . 6
5 Avancement par bloc . . . 6
6 Affectation des broches dans la version actuelle du code . . . 9
7 Configuration du bus et du périphérique I2C . . . 10
8 Correspondance entre les sorties du PCF8574 et le LCD . . . 10
9 Exemple : commande « effacer l'écran » (`0x01`, RS=0), soit quatre transactions I2C 11
10 Séquence d'initialisation du LCD . . . 11
11 Disposition des touches (table `keymap`) . . . 12
12 Paramètres du serveur BLE . . . 14
13 Tâches FreeRTOS du système . . . 15
14 Synthèse des tests BLE avec nRF Connect . . . 15
15 Problèmes rencontrés et solutions retenues . . . 17
16 Risques identifiés et pistes de traitement . . . 18
17 Évolution proposée de l'interface GATT . . . 20

1 Introduction

1.1 Contexte et objectif

Le projet vise à concevoir un dispositif d'authentification par code pour contrôler l'ouverture d'une barrière d'immeuble. Le scénario d'usage est le suivant :

1. l'habitant saisit son code sur un clavier matriciel ;
2. les caractères s'affichent en temps réel sur un écran LCD ;
3. le code complet est transmis en BLE au Raspberry Pi (serveur central) ;
4. le Raspberry Pi vérifie le code dans sa base de données et renvoie une décision ;
5. l'ESP32 affiche le résultat et, à terme, commande un servomoteur qui lève la barrière.

Au-delà du produit, le projet est un support d'apprentissage des compétences recherchées en développement embarqué : bus I2C au niveau protocole, ordonnancement temps réel (FreeRTOS), pile Bluetooth Low Energy (GATT) et intégration matériel/logiciel.

1.2 Cahier des charges

Réf.	Exigence	État
F1	Saisie d'un code de 4 caractères sur keypad 4×4	**Terminé**
F2	Affichage des caractères en temps réel sur le LCD	**Terminé**
F3	Transmission du code au Raspberry Pi par BLE	**Partiel** (lecture par téléphone validée)
F4	Vérification du code dans une base de données (sur le Raspberry Pi)	**À faire**
F5	Retour de la décision à l'ESP32 et affichage du résultat	**Partiel** (affichage à fiabiliser)
F6	Commande du servomoteur de la barrière	**À faire**
NF1	Accès concurrent sûr au LCD / bus I2C (mutex)	**En cours**
NF2	Liaison BLE sécurisée (appairage, chiffrement)	**À faire**
NF3	Données des habitants conservées uniquement sur le Raspberry Pi	**À faire**

Table 1 – Exigences fonctionnelles (F) et non fonctionnelles (NF)

1.3 Architecture globale

Le système repose sur deux nœuds : l'ESP32 côté utilisateur (acquisition, affichage, actionneur) et le Raspberry Pi côté serveur (vérification et mémoire des accès). La liaison est assurée en BLE ; le Wi-Fi du Raspberry Pi reste libre pour l'accès SSH de développement.

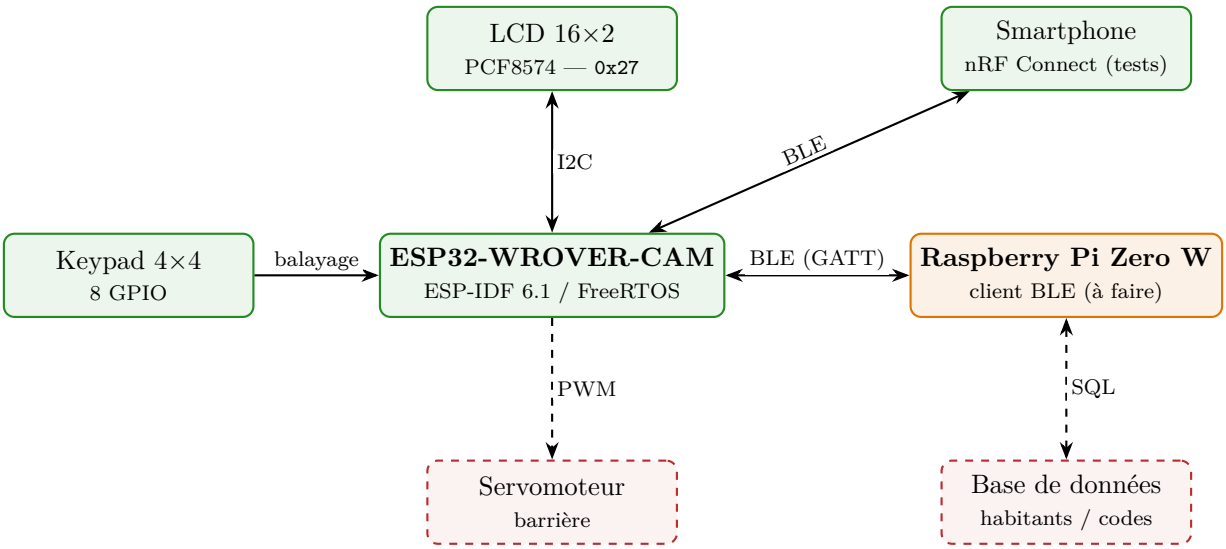


Figure 1 – Architecture globale. Contour plein vert : réalisé et testé ; orange : en cours ; rouge pointillé : à faire

1.4 Matériel et environnement logiciel

Composant	Rôle	Interface
ESP32-WROVER-CAM (Freenove)	Microcontrôleur principal, bi-cœur 240 MHz, BLE	—
LCD 1602A V2.0 + PCF8574	Afficheur 16 colonnes × 2 lignes, adresse `0x27`	I2C
Keypad matriciel 4×4	Saisie du code (16 touches)	8 GPIO
Raspberry Pi Zero W	Serveur de vérification, base de données	BLE (client)
Servomoteur	Levée de la barrière	PWM
Smartphone + nRF Connect	Outil de test du serveur GATT	BLE

Table 2 – Matériel utilisé

Outil	Détail
ESP-IDF	Version 6.1, installée avec l'ESP-IDF Installation Manager (EIM) dans `~/.espressif`
Chaîne de compilation	GCC Xtensa 15.2.0 (`-std=gnu23`), CMake, Ninja
Éditeur	VS Code avec l'extension officielle Espressif
Pile Bluetooth	NimBLE (host BLE seul), activée via `idf.py menuconfig`
Système temps réel	FreeRTOS (intégré à ESP-IDF)
Gestion de version	Git / GitHub

Table 3 – Environnement logiciel

2 État d'avancement

Progression globale : 52,5 %



L'avancement global est la moyenne simple des dix tâches ci-dessous, chacune pesant 10 %. Le paramètre `\mutexprog` (début du fichier LaTeX) permet d'ajuster la valeur du mutex sans toucher au reste.

Tâche	Statut	Avancement
Bloc A — Fondations côté ESP32		
Développement du protocole I2C	Terminé	100 %
Programmation du LCD	Terminé	100 %
Mise en place de la pile BLE	Terminé	100 %
Gestion des tâches (keypad / BLE)	Terminé	100 %
Communication ESP32 ↔ téléphone	Terminé	100 %
Bloc B — Fiabilisation et intégration du Raspberry Pi		
Gestion des tâches par mutex	En cours	25 %
Connexion du Raspberry Pi au BLE	À faire	0 %
Base de données des habitants	À faire	0 %
Communication RPi ↔ ESP32	À faire	0 %
Vérification des codes et commandes	À faire	0 %

Table 4 – Avancement détaillé par tâche

Bloc	Tâches	Avancement
A — Fondations ESP32	5	100 %
B — Fiabilisation et Raspberry Pi	5	5 %

Table 5 – Avancement par bloc

3 Travaux réalisés

3.1 Environnement de développement ESP-IDF

Le projet a d'abord été envisagé avec PlatformIO avant de passer à l'ESP-IDF natif, plus proche des pratiques industrielles et donnant accès à toutes les API Espressif. L'installation a été faite avec l'EIM (mode « Easy Installation ») après installation des prérequis système (`flex`, `bison`, `gperf`, `ccache`, `dfu-util`, `cmake`).

Deux contraintes pratiques ont structuré l'installation :

— **pas d'espace dans les chemins** : CMake refuse un chemin de projet contenant un espace (erreur « *whitespace character in project paths* ») ;

— **espace disque** : la partition racine n'avait que 2,8 Go libres pour 2,32 Go requis ; l'installation a donc été placée dans `/home` (51 Go libres).

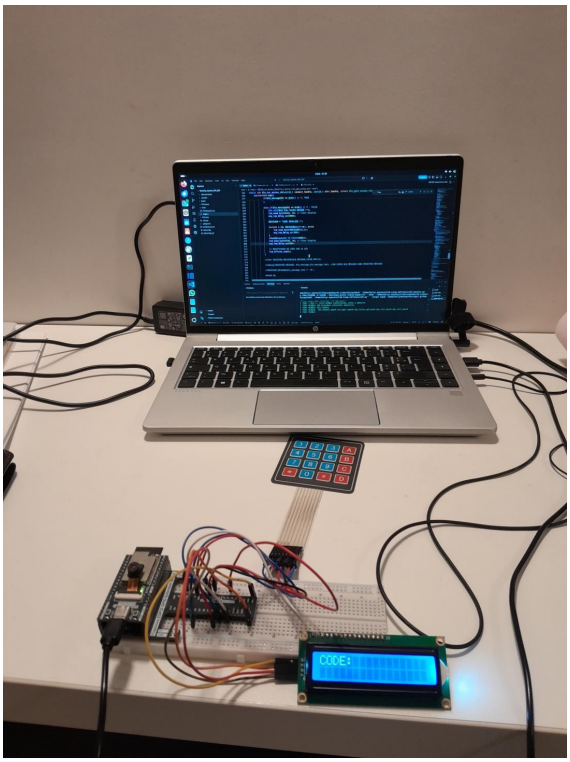


Figure 2 – Poste de travail : PC sous Ubuntu (VS Code + ESP-IDF), ESP32 sur breadboard, keypad et LCD

3.2 Câblage et affectation des broches

La carte Freenove ESP32-WROVER-CAM partage de nombreux GPIO avec le capteur caméra (signaux `CAM_Y2`...`CAM_Y9`, `CAM_XCLK`, `CAM_PCLK`...) et réserve les GPIO 6 à 11 à la mémoire flash. Le choix des broches a donc été fait à partir du brochage constructeur.

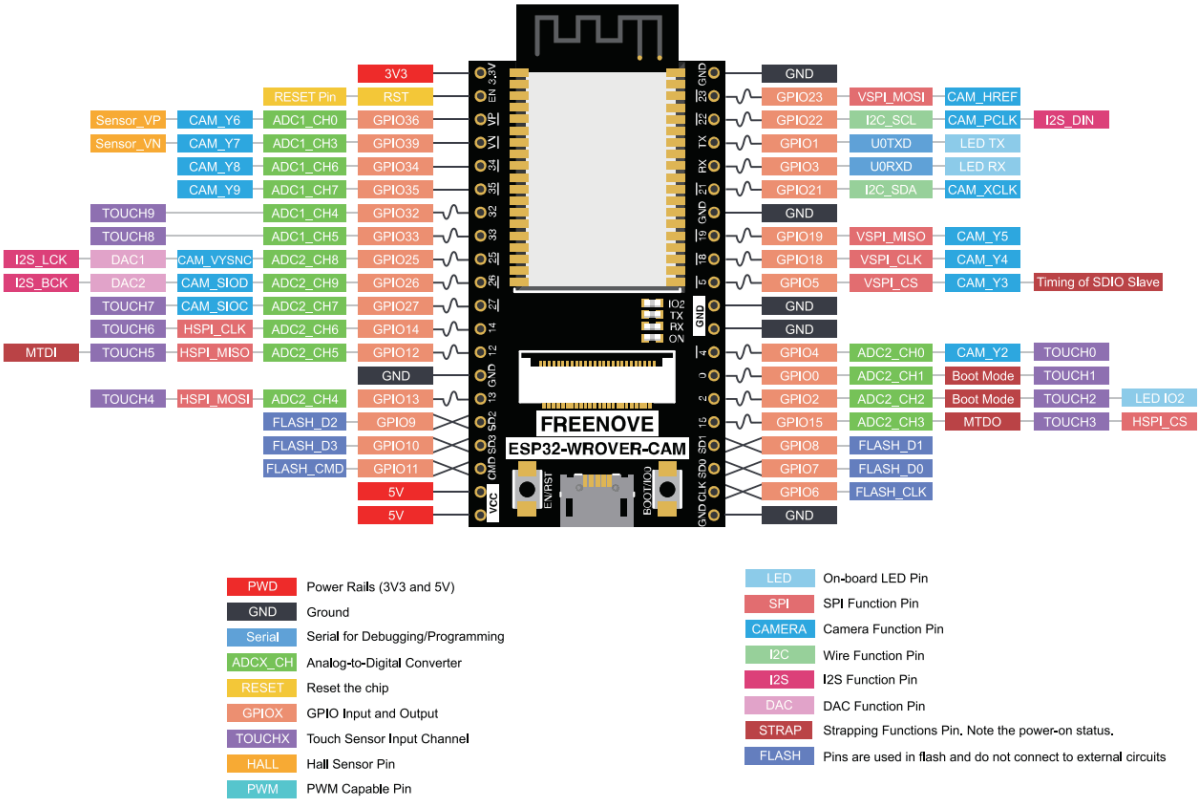


Figure 3 – Brochage de la Freenove ESP32-WROVER-CAM et fonctions de chaque broche

Pin	Function	Description
VIN	Power Input	Accepts 3.3V input for powering the board.
GND	Ground	Common ground for the circuit.
GPIO0	General Purpose I/O, Boot Mode	Used for boot mode selection and general-purpose input/output.
GPIO2	General Purpose I/O, ADC, PWM	Can be used for analog input, PWM, or digital I/O.
GPIO4	General Purpose I/O, ADC, PWM	Supports analog input, PWM, or digital I/O.
GPIO12	General Purpose I/O, ADC, PWM	Multi-function pin for analog input, PWM, or digital I/O.
GPIO13	General Purpose I/O, ADC, PWM	Multi-function pin for analog input, PWM, or digital I/O.
GPIO21	I2C SDA	Data line for I2C communication.
GPIO22	I2C SCL	Clock line for I2C communication.
GPIO23	SPI MOSI	Master Out Slave In for SPI communication.
GPIO18	SPI SCK	Clock line for SPI communication.
GPIO19	SPI MISO	Master In Slave Out for SPI communication.
GPIO16	UART RX	Receive pin for UART communication.
GPIO17	UART TX	Transmit pin for UART communication.

Note: Some GPIO pins have specific restrictions or are reserved for internal functions. Refer to the official datasheet for detailed pin mappings.

Figure 4 – Fonctions des principales broches (documentation tierce, à recouper avec le brochage ci-dessus ; les GPIO 16 et 17 ne sont pas disponibles sur WROVER, ils servent à la PSRAM)

Signal	GPIO	Mode	Remarque
SDA (I2C)	21	Bus I2C	Porte aussi `CAM_XCLK`
SCL (I2C)	22	Bus I2C	Porte aussi `CAM_PCLK`
Ligne 1 du keypad	12	Sortie	Libre
Ligne 2 du keypad	13	Sortie	Libre
Ligne 3 du keypad	14	Sortie	Libre
Ligne 4 du keypad	32	Sortie	Libre
Colonne 1 du keypad	0	Entrée + pull-up	Broche de *strapping* (mode de démarrage)
Colonne 2 du keypad	5	Entrée + pull-up	Partagée avec `CAM_Y3`
Colonne 3 du keypad	18	Entrée + pull-up	Partagée avec `CAM_Y4`
Colonne 4 du keypad	19	Entrée + pull-up	Partagée avec `CAM_Y5`

Table 6 – Affectation des broches dans la version actuelle du code

Broches partagées avec la caméra

La caméra n'est *jamais initialisée* dans ce projet (aucun appel à `esp_camera_init()`) : aucune horloge `XCLK` n'est générée et le capteur reste passif. Aucun conflit n'a été observé sur l'I2C. Sur le keypad, la colonne 1 a demandé plusieurs essais de broches (4, 33, 26 puis 0) ; la validation finale de cette affectation reste à faire (voir section 5).

3.3 Protocole I2C

3.3.1 Principe

L'I2C est un bus série synchrone à deux fils : **SDA** (données) et **SCL** (horloge, générée par le maître). Les deux lignes sont en *drain ouvert* : un équipement ne peut que tirer la ligne vers le bas ; c'est une **résistance de rappel (pull-up)** qui la ramène au niveau haut au repos. Ce principe évite tout court-circuit entre équipements et autorise l'arbitrage multi-maître, l'accusé de réception (ACK) par l'esclave et l'étirement d'horloge. Chaque esclave est identifié par une adresse sur 7 bits.

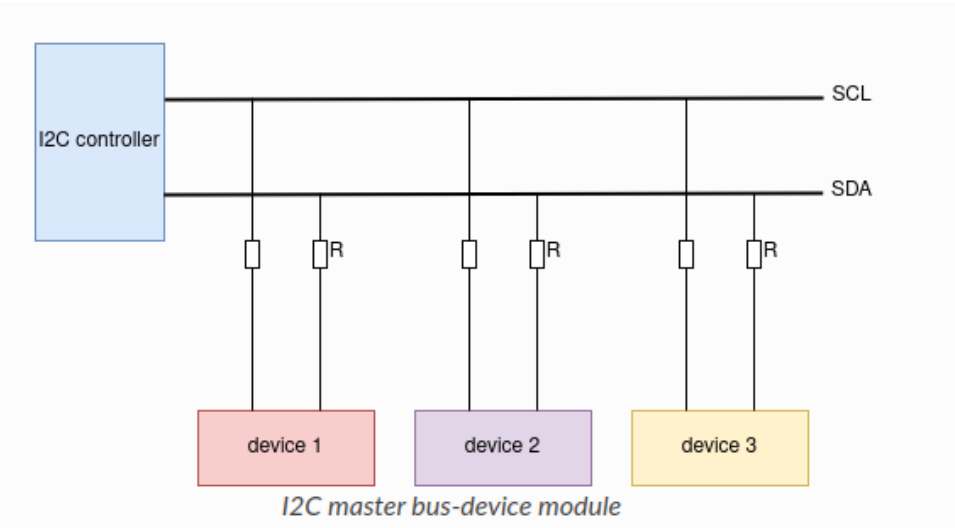


Figure 5 – Principe du bus I2C maître / esclaves avec résistances de rappel (d'après la documentation ESP-IDF)

3.3.2 Mise en œuvre avec l'API `i2c_master` d'ESP-IDF

Le pilote est écrit avec la nouvelle API d'ESP-IDF, organisée en trois niveaux : le **bus** (`i2c_new_master_bus`), le **périphérique** ajouté sur ce bus (`i2c_master_bus_add_device`) et les **transactions** (`i2c_master_transmit`, `i2c_master_probe`).

Paramètre	Valeur
Contrôleur	`I2C_NUM_0`
SDA / SCL	GPIO 21 / GPIO 22
Fréquence SCL	100 kHz (mode standard)
Filtre anti-parasites	7 cycles (`glitch_ignore_cnt`)
Pull-up interne	Activé (en renfort de ceux du module LCD)
Adresse du LCD	`0x27`, 7 bits
Délai d'attente d'étirement	1000 μs (`scl_wait_us`)

Table 7 – Configuration du bus et du périphérique I2C

3.3.3 Balayage du bus

Une fonction de scan parcourt les adresses `0x08` à `0x77` (plage utilisable en 7 bits) avec `i2c_master_probe()` et un délai de 50 ms par adresse. Elle valide à la fois le câblage et l'adresse du module, sans la deviner.

```
W (254) spi_flash: Detected size(4096k) larger than the size in the binary image header(2048k). Using the size in the binary image header.
I (268) main_task: Started on CPU0
I (268) main_task: Calling app_main()
I (268) I2C_DRIVER: I2C BUS SUCCESSFULLY CREATED (SDA = 21, SCL = 22)
I (278) I2C_DRIVER: DEVICE FOUND AT THE ADDRESS :27
I (288) main_task: Returned from app_main()
```

Figure 6 – Journal du moniteur : bus créé, périphérique détecté à l'adresse `0x27`

3.4 Programmation du LCD

3.4.1 Rôle du PCF8574

Le contrôleur HD44780 du LCD attend des signaux parallèles. Le module intègre un **PCF8574**, expandeur de ports qui recopie chaque octet reçu en I2C sur ses huit sorties P0 à P7, câblées en dur vers le LCD. Le LCD est piloté en **mode 4 bits** : seules D4 à D7 sont disponibles, car les autres sorties servent au contrôle.

Broche	Reliée à	Rôle
P0	RS	Sélection commande (0) / donnée (1)
P1	RW	Lecture / écriture (forcée à 0 : écriture seule)
P2	E	Enable : le LCD lit ses entrées sur le front descendant
P3	Rétroéclairage	Allumé (1) ou éteint (0)
P4 à P7	D4 à D7	Quatre bits de donnée (mode 4 bits)

Table 8 – Correspondance entre les sorties du PCF8574 et le LCD

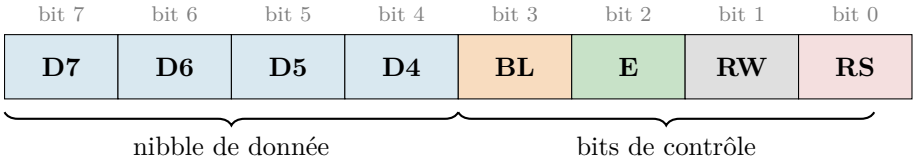


Figure 7 – Structure de l'octet envoyé au PCF8574

3.4.2 Envoi d'un octet au LCD

Un octet destiné au LCD (commande ou caractère) est découpé en deux *nibbles*. Chaque nibble est envoyé en deux transactions I2C : une avec **E=1**, puis la même avec **E=0**. Le front descendant de E déclenche la lecture par le LCD. L'octet I2C se construit ainsi :

```
octet = (nibble « 4) | (1 « 3) | (1 « 2) | rs   (E=1)
octet = (nibble « 4) | (1 « 3) | rs   (E=0)
```

Ordre	Nibble	Octet I2C	Signification
1	`0000`	`0x0C`	Nibble haut, E=1
2	`0000`	`0x08`	Nibble haut, E=0 (lecture)
3	`0001`	`0x1C`	Nibble bas, E=1
4	`0001`	`0x18`	Nibble bas, E=0 (lecture)

Table 9 – Exemple : commande « effacer l'écran » (`0x01`, RS=0), soit quatre transactions I2C

Trois fonctions ont été écrites : `lcd_write_nibble()` (un nibble avec la bascule de E), `lcd_send_byte()` (découpage haut / bas) et `lcd_init()` (initialisation).

3.4.3 Séquence d'initialisation du HD44780

N°	Action	Rôle / délai
1	Attente de 50 ms	Stabilisation après mise sous tension
2	Nibble `0x3` (3 fois)	Réveil du contrôleur ; 4,5 ms, 4,5 ms puis 150 μs
3	Nibble `0x2`	Passage en mode 4 bits
4	Octet `0x28`	Function Set : 4 bits, 2 lignes, police 5×8
5	Octet `0x0C`	Affichage activé, curseur masqué
6	Octet `0x01` + 2 ms	Effacement (commande lente)
7	Octet `0x06`	Curseur auto-incrémenté vers la droite

Table 10 – Séquence d'initialisation du LCD

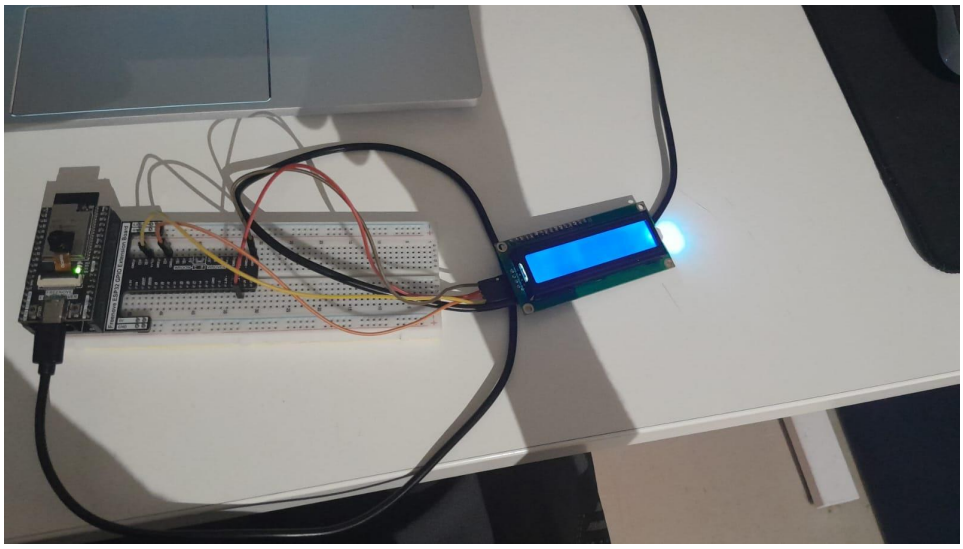


Figure 8 – Premier montage ESP32 + LCD I2C, rétroéclairage actif

Alimentation et contraste du module LCD

Le module n'affichait rien tant que son alimentation n'était pas adaptée (ce type de module est prévu pour 5 V) ; le contraste se règle ensuite avec le petit potentiomètre bleu du dos du module. L'I2C répondait pourtant correctement dès le départ (ACK à `0x27`) : l'absence d'affichage ne prouve donc pas une panne de communication.

3.5 Keypad matriciel 4×4

3.5.1 Principe du balayage

Un keypad 4×4 ne câble pas 16 boutons indépendants : chaque touche relie une **ligne** à une **colonne**. Huit GPIO suffisent donc pour seize touches. Les lignes sont pilotées en sortie par l'ESP32, les colonnes lues en entrée avec pull-up (niveau haut au repos).

1. toutes les lignes sont mises à l'état haut ;
2. une seule ligne est abaissée à l'état bas ;
3. les quatre colonnes sont lues : une colonne à l'état bas indique qu'une touche de cette ligne est pressée (le niveau bas traverse le contact) ;
4. la procédure est répétée pour chaque ligne.

Abaisser une seule ligne à la fois lève toute ambiguïté : si une colonne passe à l'état bas, la touche pressée se trouve forcément à l'intersection de cette colonne et de la ligne testée. Le caractère est ensuite retrouvé dans la table `keymap[ligne][colonne]`.

	C1	**C2**	**C3**	**C4**
L1	1	2	3	A
L2	4	5	6	B
L3	7	8	9	C
L4	*	0	#	D

Table 11 – Disposition des touches (table `keymap`)

3.5.2 Anti-rebond et tâche FreeRTOS

Les contacts mécaniques rebondissent pendant quelques millisecondes ; sans précaution, un appui serait compté plusieurs fois. Dès qu'une colonne passe à l'état bas, le code attend 20 ms puis relit l'entrée ; l'appui n'est validé que si elle est toujours basse, puis le programme attend le relâchement de la touche. Le balayage s'exécute dans une tâche dédiée `keypad_task` (pile de 2048 octets, priorité 5), cadencée par un `vTaskDelay` de 10 ms qui laisse la main aux autres tâches.

Keypad	GPIO	Justification
L1 (sortie)	GPIO 12	100% libre, aucun lien caméra
L2 (sortie)	GPIO 13	100% libre
L3 (sortie)	GPIO 14	100% libre
L4 (sortie)	GPIO 32	100% libre
C1 (entrée, pull-up)	GPIO 4	Camera Y2 — sens sortie caméra/entrée ESP32, cohéren
C2 (entrée, pull-up)	GPIO 5	Camera Y3 — même logique
C3 (entrée, pull-up)	GPIO 18	Camera Y4 — même logique
C4 (entrée, pull-up)	GPIO 19	Camera Y5 — même logique

Figure 9 – Première affectation des broches du keypad (la colonne 1 a ensuite changé de broche, voir tableau 6)



Figure 10 – Journal du moniteur série lors de l'appui successif sur les touches du keypad

3.6 Pile Bluetooth Low Energy (NimBLE)

3.6.1 Concepts GATT

En BLE, l'ESP32 joue le rôle de **serveur GATT** (périphérique) et expose ses données sous forme de **services** contenant des **caractéristiques**, identifiés par des UUID. Le client (smartphone, puis Raspberry Pi) se connecte, lit ou écrit ces caractéristiques. NimBLE est **événementielle** : la pile tourne dans sa propre tâche FreeRTOS et *appelle* le code de l'application (callbacks) lorsqu'un événement survient.

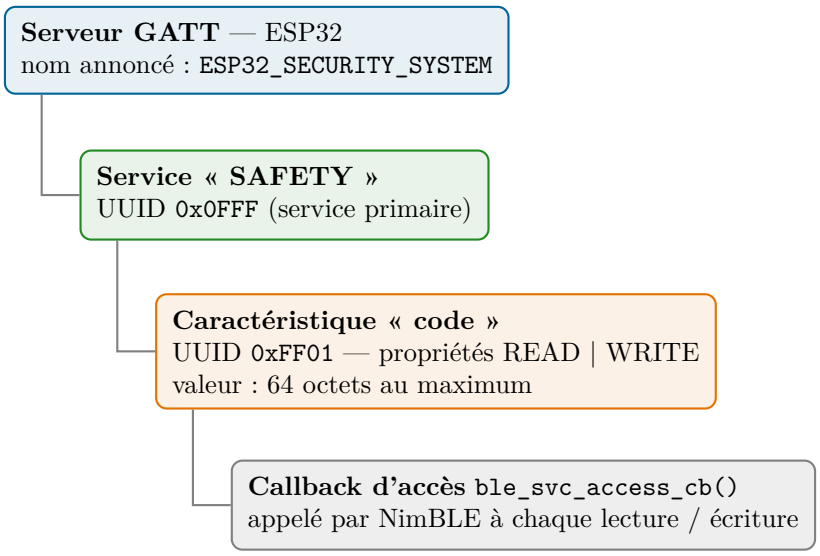


Figure 11 – Hiérarchie GATT du projet

3.6.2 Démarrage de la pile

La séquence d'initialisation de `app_main()` est la suivante :

1. `nvs_flash_init()` (effacement et nouvel essai en cas de zone pleine ou de version incompatible) : NimBLE a besoin de la mémoire non volatile ;
2. `nimble_port_init()` : initialise les structures internes de la pile ;
3. enregistrement du callback de synchronisation `ble_hs_cfg.sync_cb` ;
4. `ble_gatt_svr_init()` : services GAP et GATT standards, puis enregistrement du service du projet ;
5. `nimble_port_freertos_init()` : lance la tâche FreeRTOS qui exécute `nimble_port_run()`.

L'*advertising* (annonce radio) ne peut démarrer qu'une fois la pile synchronisée avec le contrôleur radio. Il est donc lancé *dans* le callback `ble_app_on_sync()` et non dans `app_main()`.

Paramètre	Valeur
Pile / rôle	NimBLE (host seul) / périphérique, serveur GATT
Nom annoncé	`ESP32_SECURITY_SYSTEM`
Service	UUID `0x0FFF`, primaire
Caractéristique	UUID `0xFF01`, READ + WRITE
Taille maximale de la valeur	64 octets
Mode de connexion	Non dirigé (tout client peut se connecter)
Découvrabilité	Générale, annonce permanente
Sécurité	Aucune (affichée *NOT BONDED*) — à renforcer

Table 12 – Paramètres du serveur BLE

3.6.3 Callback d'accès

La fonction `ble_svc_access_cb()` est enregistrée sur la caractéristique. En lecture, elle recopie le tampon `ble_message` dans la réponse (`os_mbuf_append`). En écriture, elle récupère les octets envoyés par le client (`ble_hs_mbuf_to_flat`), contrôle la longueur et les stocke. Les octets

reçus arrivent d'abord dans une structure interne de NimBLE (*mbuf*, `ctxt->om`), que le code convertit en tableau d'octets simple.

3.7 Gestion des tâches et saisie du code

Contexte d'exécution	Responsabilité
`app_main()`	Initialisation (I2C, LCD, NVS, NimBLE) puis création des tâches
`keypad_task`	Balayage du keypad toutes les 10 ms, affichage des touches sur le LCD, remplissage du tampon BLE
Tâche hôte NimBLE	Pile BLE ; exécute le callback d'accès à chaque lecture / écriture du client

Table 13 – Tâches FreeRTOS du système

La saisie fonctionne par un index `char_index`. Chaque touche valide est affichée sur le LCD (précédée du texte « CODE : »), recopiée dans `ble_message[char_index]`, puis la longueur publiée vaut `++char_index`. Au quatrième caractère, l'index est remis à zéro pour la saisie suivante, sans attendre une cinquième touche. Le code complet est alors lisible par un client BLE.

Protocole applicatif actuel

Sens	Contenu	Format
ESP32 → client (lecture)	Code saisi	4 caractères ASCII (ex. `2A3B`)
Client → ESP32 (écriture)	Décision	1 octet : `'T'` (`0x54`) accès accordé, `'F'` (`0x46`) refusé

À la réception d'une décision, l'ESP32 efface le LCD et affiche « CODE VALIDE ! » ou « CODE INVALIDE ! ».

3.8 Communication ESP32 ↔ téléphone

Le serveur GATT a été testé avec l'application **nRF Connect** : l'ESP32 est détecté sous le nom `ESP32_SECURITY_SYSTEM`, la connexion s'établit et les services découverts sont *Generic Access* (`0x1800`), *Generic Attribute* (`0x1801`) et le service du projet (`0x0FFF`) avec sa caractéristique `0xFF01` en READ / WRITE.

Test	Action	Résultat observé	Statut
Lecture initiale	Lecture de la caractéristique	11 octets renvoyés (« ESP-READY ! »), journal de lecture sur l'ESP32	**Terminé**
Écriture d'un octet	Envoi de `0x45`	1 octet reçu, message « E » affiché dans le journal	**Terminé**
Écriture de texte	Chaînes de test de 10 et 11 octets	Octets reçus à l'identique (hexadécimal et texte concordent)	**Terminé**
Code du keypad	Saisie de « 2A3B », puis lecture	Valeur lue `32-41-33-42` = « 2A3B »	**Terminé**
Décision → LCD	Écriture de `'T'` / `'F'`	Message affiché, mais caractères parasites parfois visibles	**Partiel**

Table 14 – Synthèse des tests BLE avec nRF Connect

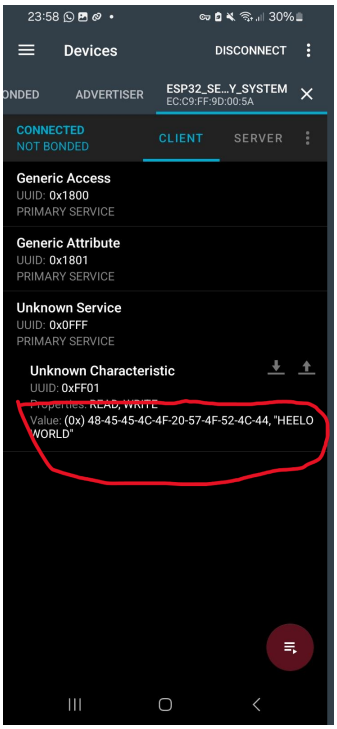


Figure 12 – Service et caractéristique vus par nRF Connect, après écriture d'une chaîne de test

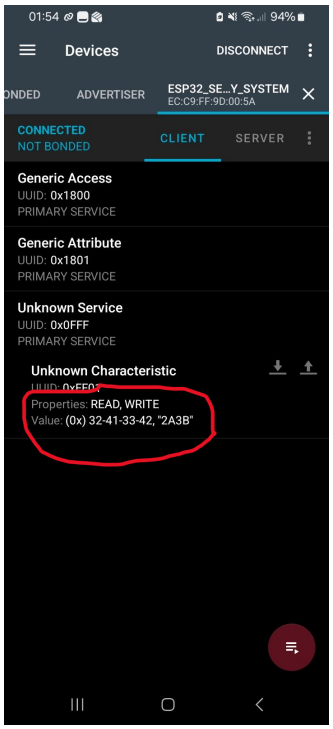


Figure 13 – Lecture par le téléphone du code « 2A3B » saisi au keypad

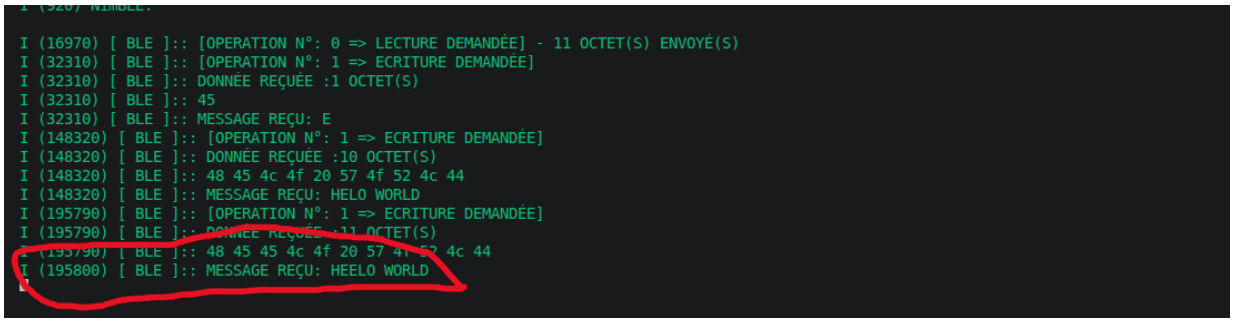


Figure 14 – Journal de l'ESP32 : lecture de 11 octets, écriture d'un octet puis de chaînes de test, réception à l'identique

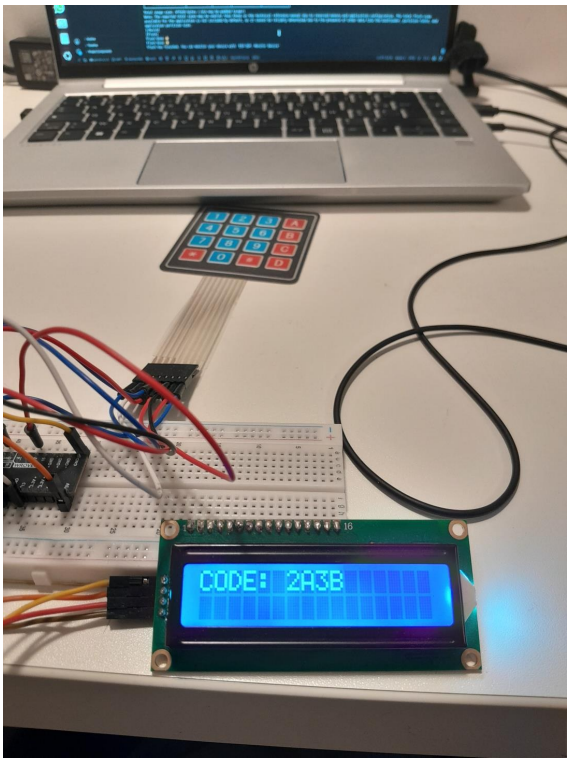


Figure 15 – Le code « 2A3B » saisi au keypad, affiché en temps réel sur le LCD et lu par le téléphone

4 Difficultés rencontrées et solutions

Table 15 – Problèmes rencontrés et solutions retenues

Problème	Cause	Solution
Compilation refusée dès la création du projet	Espace dans le chemin du projet : CMake ne le supporte pas	Chemin sans espace
Installation d'ESP-IDF bloquée	Prérequis absents (`flex`, `bison`, `gperf`, `ccache`, `dfu-util`, `cmake`)	Installation via `apt`
Espace disque insuffisant	2,8 Go libres sur la racine pour 2,32 Go d'installation	Installation dans `/home`
LCD sans affichage alors que l'I2C répond	Alimentation du module inadaptée, puis contraste mal réglé	Alimentation corrigée et réglage du potentiomètre
Colonne 1 du keypad inactive, caractères parasites	Broches partagées avec la caméra, contacts de breadboard peu fiables	Changements de broches ; pas de caméra initialisée ; validation à refaire
Cascade d'erreurs de compilation	Un point-virgule manquant faisait lire deux déclarations comme une seule	Corriger la première erreur du journal avant toute autre
Erreurs « implicit declaration »	En-têtes NimBLE manquants	Ajout de `nimble_port_freertos.h`, `host/ble_hs.h`, `services/gap/ble_svc_gap.h`, `services/gatt/ble_svc_gatt.h`
Une écriture BLE journalisée comme une lecture	Confusion entre les constantes de la couche ATT (`BLE_ATT_ACCESS_OP_*`, valeurs 1 et 2) et celles du GATT	Utiliser `BLE_GATT_ACCESS_OP_READ_CHR` et `BLE_GATT_ACCESS_OP_WRITE_CHR` (valeurs 0 et 1)

Problème	Cause	Solution
Journal affichant « (null) » ou « AAAAREADY ! »	Arguments de `ESP_LOGI` dans le désordre ; chaîne non terminée par `'\0'`	Respecter l'ordre des `%` ; ajouter le `'\0'` ou utiliser `%.*s`
Saisie décalée d'un caractère	Longueur publiée avant l'incrément ; index remis à zéro à la 5e touche	Pré-incrément `++char_index` ; remise à zéro dès le 4e caractère
Push GitHub refusé	Un jeton d'accès avait été collé dans `main.c` ; la protection de GitHub l'a détecté	Jeton révoqué, historique réinitialisé ; ne jamais committer de secret
Caractères parasites sur le LCD à l'affichage de la décision	Deux tâches (keypad et callback BLE) écrivent sur le bus I2C en même temps	Mutex (en cours, voir section 6.1)

5 Analyse de sécurité et points d'attention

Le projet protège un accès physique : la sécurité de la liaison BLE n'est pas un détail. L'état actuel est celui d'un prototype de laboratoire, et plusieurs points doivent être traités avant tout usage réel.

Table 16 – Risques identifiés et pistes de traitement

Risque	Niveau	Description	Piste
Liaison BLE non appairée	**Critique**	Tout appareil à portée peut se connecter et écrire `'T'` dans la caractéristique : la décision d'ouverture n'est pas authentifiée	Appairage avec chiffrement (LE Secure Connections), caractéristiques exigeant un lien chiffré, liste blanche de l'adresse du Raspberry Pi
Code transmis en clair	**Élevé**	Le code circule sans chiffrement et la caractéristique peut être lue par n'importe quel client	Lien chiffré ; ne jamais conserver le code en clair côté Raspberry Pi (stocker une empreinte salée)
Essais illimités	**Moyen**	Un code de 4 caractères sur 16 symboles offre au plus 65 536 combinaisons	Limitation du nombre d'essais, temporisation croissante, journal des accès
Tampon BLE	**Moyen**	Voir encadré ci-dessous : risques de lecture hors limites et de données incohérentes	Longueur locale, contrôle strict, protection par verrou
GPIO 0 (colonne 1)	**Moyen**	Broche de *strapping* : un niveau bas au démarrage fait entrer l'ESP32 en mode téléchargement	Déplacer la colonne sur une broche libre, ou vérifier qu'aucune touche n'est tenue au démarrage
Niveaux logiques	**Moyen**	Les GPIO de l'ESP32 sont en 3,3 V alors que le module LCD est alimenté en 5 V ; ses résistances de rappel sont en général reliées à son alimentation	Vérifier la tension sur SDA/SCL ; au besoin, adaptateur de niveau
Taille de flash	**Faible**	Avertissement au démarrage : flash de 4 Mo détectée, 2 Mo déclarés dans l'image	Régler la taille de flash dans `menuconfig`

Risque	Niveau	Description	Piste
Secrets dans le dépôt Git	Traité	Un jeton avait été commité par erreur	Révocation, historique réinitialisé, fichier d'exclusion pour les secrets

Dette technique à corriger dans le callback BLE

1. En écriture, `ble_message_len` est écrasé *avant* le contrôle de longueur : une valeur trop longue laisse une longueur supérieure à 64, et la lecture suivante dépasserait le tampon. Utiliser une variable locale et ne publier la longueur qu'après validation.
2. Le contrôle autorise une longueur de 64 octets, puis le programme écrit `'\0'` à l'indice 64, hors du tableau de 64 octets. Refuser les longueurs ≥ 64.
3. `keypad_task` et le callback BLE modifient `ble_message` et `ble_message_len` sans protection : ces variables doivent être couvertes par le même mécanisme d'exclusion que le LCD.
4. Les constantes numériques `0` et `1` utilisées dans le `switch` gagnent à être remplacées par les noms `BLE_GATT_ACCESS_OP_READ_CHR` et `BLE_GATT_ACCESS_OP_WRITE_CHR`.

6 Travaux restants

6.1 Mise en place d'un mutex

Le LCD et le bus I2C sont des ressources partagées. Le callback BLE, exécuté dans la tâche NimBLE, affiche la décision pendant que `keypad_task` peut écrire un caractère au même moment : les transactions I2C s'entremêlent et produisent les caractères parasites observés. Les délais supplémentaires n'ont rien changé, ce qui confirme que la cause est la concurrence et non le temps d'exécution des commandes du LCD.

— créer un mutex FreeRTOS (`xSemaphoreCreateMutex`) avant le lancement des tâches ;

— regrouper les envois au LCD dans des fonctions (`lcd_print()`, `lcd_clear()`) qui prennent le mutex *une seule fois pour toute une séquence* (et non octet par octet) ;

— protéger aussi `ble_message` et sa longueur ;

— ne pas tenir le mutex pendant une attente longue (priorité à la brièveté de la section critique) ;

— variante à évaluer : le callback BLE ne touche plus au LCD, il dépose la décision dans une file (*queue*) lue par une tâche d'affichage unique.

Critère de réussite : saisie au keypad pendant l'envoi répété de décisions depuis le téléphone, sans aucun caractère altéré.

6.2 Connexion du Raspberry Pi au BLE de l'ESP32

Le Raspberry Pi Zero W dispose du Bluetooth 4.1 (compatible BLE) géré par la pile BlueZ. Il jouera le rôle de **client GATT** :

— balayage des appareils BLE et identification de l'ESP32 par son nom ou son UUID de service ;

— connexion, découverte du service `0x0FFF` et de ses caractéristiques ;

— abonnement aux notifications, afin d'éviter l'interrogation permanente ;

— reconnexion automatique en cas de coupure.

Une bibliothèque Python asynchrone de type *bleak* est une piste à valider sur ce matériel. Côté ESP32, la caractéristique du code devra gagner la propriété NOTIFY.

6.3 Base de données des habitants

Elle sera hébergée sur le Raspberry Pi, qui reste le maître : aucune donnée d'habitant ne sera stockée sur l'ESP32. Une base SQLite (fichier local, sans serveur) suffit pour cet usage.

Listing 1 – Schéma envisagé

```
CREATE TABLE habitants (
    id          INTEGER PRIMARY KEY,
    nom         TEXT NOT NULL,
    appartement TEXT NOT NULL,
    code_hash   TEXT NOT NULL,   -- empreinte salee, jamais le code en clair
    sel         TEXT NOT NULL,
    actif       INTEGER NOT NULL DEFAULT 1
);

CREATE TABLE acces (
    id          INTEGER PRIMARY KEY,
    horodatage  TEXT NOT NULL,
    habitant_id INTEGER REFERENCES habitants(id),   -- NULL si code inconnu
    resultat    TEXT NOT NULL                       -- 'accepte' ou 'refuse'
);
```

6.4 Communication RPi ↔ ESP32

Aujourd'hui, une seule caractéristique sert dans les deux sens (lecture du code, écriture de la décision). Pour fiabiliser l'échange, deux caractéristiques distinctes sont prévues :

Caractéristique	UUID	Propriétés	Sens
Code saisi	`0xFF01`	READ + NOTIFY	ESP32 → Raspberry Pi
Décision	`0xFF02` (proposé)	WRITE	Raspberry Pi → ESP32

Table 17 – Évolution proposée de l'interface GATT

6.5 Vérification des codes et envoi de commandes

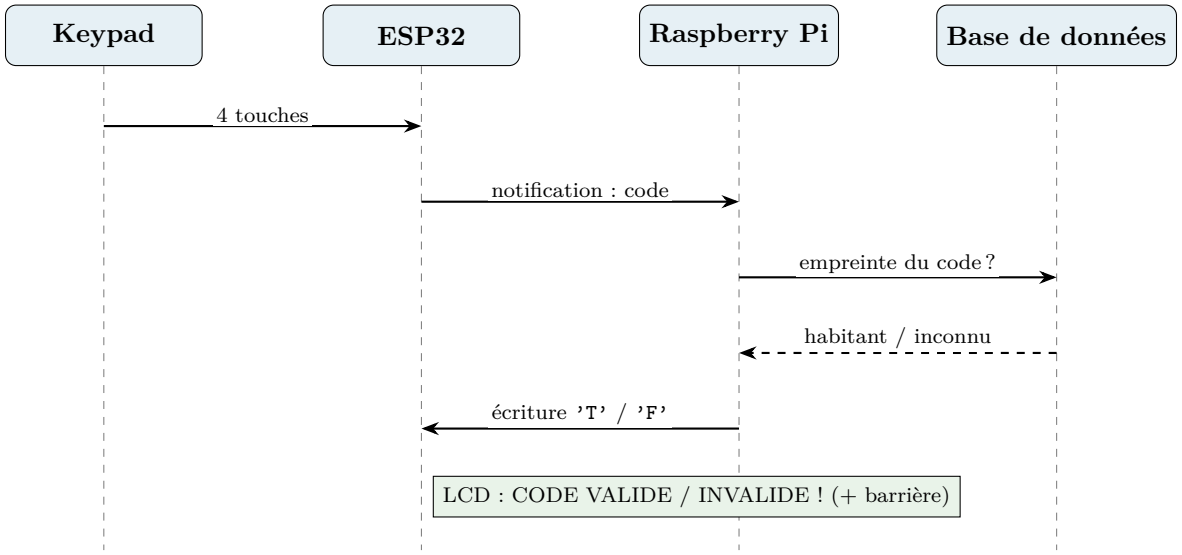


Figure 16 – Séquence cible de vérification d'un code

Le Raspberry Pi comparera l'empreinte du code reçu à celles de la base, journalisera chaque tentative et renverra `'T'` ou `'F'`. Une limitation du nombre d'échecs consécutifs sera ajoutée (voir section 5).

6.6 Perspectives

— **Barrière** : commande d'un servomoteur par PWM à 50 Hz (périphérique LEDC), sur une broche libre, avec une alimentation 5 V dédiée et une masse commune avec l'ESP32 ;

— **Sécurisation BLE** : appairage, chiffrement, liste blanche ;

— **Ergonomie LCD** : effacer l'écran et réafficher « CODE : » après chaque validation, masquer les caractères saisis ;

— **Livrables de présentation** : maquette 3D d'un immeuble de trois étages avec implantation de l'ESP32 et du Raspberry Pi, puis vidéo de démonstration avec légendes sur le rôle de chaque composant.

6.7 Feuille de route

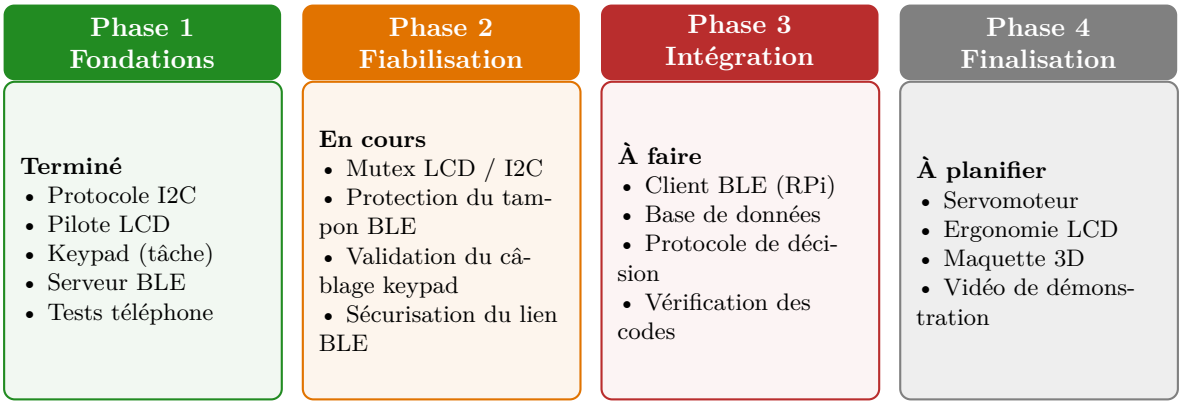


Figure 17 – Feuille de route du projet

7 Conclusion

Le projet a franchi une étape importante : la chaîne locale saisie → affichage → transmission fonctionne. Le pilote I2C et le pilote LCD ont été écrits de bout en bout (adressage, mode 4 bits, bascule du signal E, séquence d'initialisation), le keypad est balayé par une tâche FreeRTOS dédiée et le serveur GATT a été validé avec un smartphone, en lecture comme en écriture.

La suite se déroule en deux temps. Il faut d'abord fiabiliser l'existant (mutex, protection du tampon BLE, validation du câblage du keypad). Il faut ensuite intégrer le Raspberry Pi (client BLE, base de données, vérification et commandes), en traitant en parallèle la sécurité de la liaison, dont l'absence actuelle d'appairage est le principal point faible. Les compétences acquises — protocole I2C au niveau registre, ordonnancement temps réel, architecture GATT — sont directement transférables à des projets embarqués professionnels.

A Extraits de code

Les extraits sont condensés par rapport au fichier source ; le callback BLE est présenté dans sa version recommandée, qui intègre les corrections de la section 5.

A.1 Bus I2C et balayage

Listing 2 – Initialisation du bus et scan

```
#define I2C_MASTER_SDA_IO  GPIO_NUM_21
#define I2C_MASTER_SCL_IO  GPIO_NUM_22
#define I2C_MASTER_NUM     I2C_NUM_0
#define I2C_MASTER_FREQ_HZ 100000

static i2c_master_bus_handle_t bus_handle;
static i2c_master_dev_handle_t lcd_handle;

esp_err_t i2c_master_init(void) {
    i2c_master_bus_config_t cfg = {
        .clk_source = I2C_CLK_SRC_DEFAULT,
        .i2c_port   = I2C_MASTER_NUM,
        .scl_io_num = I2C_MASTER_SCL_IO,
        .sda_io_num = I2C_MASTER_SDA_IO,
        .glitch_ignore_cnt = 7,
        .flags.enable_internal_pullup = true,
    };
```

```
    esp_err_t err = i2c_new_master_bus(&cfg, &bus_handle);
    if (err != ESP_OK) {
        ESP_LOGE(TAG, "FAIL TO CREATE THE I2C DRIVER: %s", esp_err_to_name(err));
        return err;
    }
    return ESP_OK;
}

void addressScan(void) {
    for (int addr = 0x08; addr <= 0x77; addr++) {
        if (i2c_master_probe(bus_handle, addr, 50) == ESP_OK)
            ESP_LOGI(TAG, "DEVICE FOUND AT THE ADDRESS :%02X", addr);
    }
}
```

A.2 Pilote du LCD

Listing 3 – Envoi d'un nibble, d'un octet et initialisation

```
void lcd_write_nibble(uint8_t nibble, uint8_t rs) {
    // bits : D7 D6 D5 D4 | BL E RW RS
    uint8_t e1 = (nibble << 4) | (1 << 3) | (1 << 2) | rs;   // E = 1
    uint8_t e0 = (nibble << 4) | (1 << 3) | rs;              // E = 0 : le LCD lit
    i2c_master_transmit(lcd_handle, &e1, 1, 100);  esp_rom_delay_us(100);
    i2c_master_transmit(lcd_handle, &e0, 1, 100);  esp_rom_delay_us(100);
}

void lcd_send_byte(uint8_t byte, uint8_t rs) {
    lcd_write_nibble(byte >> 4, rs);
    esp_rom_delay_us(100);
    lcd_write_nibble(byte & 0x0F, rs);
}

void lcd_init(void) {
    esp_rom_delay_us(50000);
    lcd_write_nibble(0x03, 0); esp_rom_delay_us(4500);
    lcd_write_nibble(0x03, 0); esp_rom_delay_us(4500);
    lcd_write_nibble(0x03, 0); esp_rom_delay_us(150);
    lcd_write_nibble(0x02, 0);                  // passage en mode 4 bits
    lcd_send_byte(0x28, 0);                     // 4 bits, 2 lignes, 5x8
    lcd_send_byte(0x0C, 0);                     // affichage ON, curseur OFF
    lcd_send_byte(0x01, 0); esp_rom_delay_us(2000);   // effacement (lent)
    lcd_send_byte(0x06, 0);                     // curseur auto-incrémenté
}
```

A.3 Keypad

Listing 4 – Balayage matriciel et tâche de saisie

```
char keypad_scan(void) {
    char key = 0;
    for (int r = 0; r < 4; r++) {
        for (int i = 0; i < 4; i++) gpio_set_level(row_pins[i], 1);
        gpio_set_level(row_pins[r], 0);              // une seule ligne à l'état bas
        vTaskDelay(pdMS_TO_TICKS(1));
        for (int c = 0; c < 4; c++) {
            if (gpio_get_level(col_pins[c]) == 0) {
                vTaskDelay(pdMS_TO_TICKS(20));        // anti-rebond
                if (gpio_get_level(col_pins[c]) == 0) {
                    key = keymap[r][c];
                    while (gpio_get_level(col_pins[c]) == 0)   // attente du relâchement
                        vTaskDelay(pdMS_TO_TICKS(10));
                }
            }
        }
    }
    return key;
}
```

```
20
21 void keypad_task(void *pv) {
22     keymap_gpio_init();
23     while (1) {
24         char key = keypad_scan();
25         if (key != 0) {
26             lcd_send_byte(key, 1);                    // affichage immédiat
27             if (char_index <= 3) {
28                 ble_message[char_index] = key;
29                 ble_message_len = ++char_index;
30                 if (char_index == 4) char_index = 0;  // code complet
31             }
32         }
33         vTaskDelay(pdMS_TO_TICKS(10));
34     }
35 }
```

A.4 Serveur BLE

Listing 5 – Advertising, définition GATT et callback d'accès (version recommandée)

```
1  #define GATT_SVC_UUID      0x0FFF
2  #define GATT_CHR_CODE_UUID 0xFF01
3  #define BLE_MESSAGE_VALUE_MAX 64
4
5  static uint8_t  ble_message[BLE_MESSAGE_VALUE_MAX] = "ESP-READY !";
6  static uint16_t ble_message_len = 11;
7
8  void ble_app_on_sync(void) {
9      ble_svc_gap_device_name_set("ESP32_SECURITY_SYSTEM");
10
11     struct ble_hs_adv_fields fields;
12     memset(&fields, 0, sizeof(fields));
13     fields.flags = BLE_HS_ADV_F_DISC_GEN | BLE_HS_ADV_F_BREDR_UNSUP;
14     fields.name = (uint8_t *)"ESP32_SECURITY_SYSTEM";
15     fields.name_len = strlen("ESP32_SECURITY_SYSTEM");
16     fields.name_is_complete = 1;
17     ble_gap_adv_set_fields(&fields);
18
19     struct ble_gap_adv_params adv;
20     memset(&adv, 0, sizeof(adv));
21     adv.conn_mode = BLE_GAP_CONN_MODE_UND;
22     adv.disc_mode = BLE_GAP_DISC_MODE_GEN;
23     ble_gap_adv_start(BLE_OWN_ADDR_PUBLIC, NULL, BLE_HS_FOREVER, &adv, NULL, NULL);
24 }
25
26 static int ble_svc_access_cb(uint16_t conn_handle, uint16_t attr_handle,
27                              struct ble_gatt_access_ctxt *ctxt, void *arg) {
28     switch (ctxt->op) {
29     case BLE_GATT_ACCESS_OP_READ_CHR:
30         return os_mbuf_append(ctxt->om, ble_message, ble_message_len) == 0
31                ? 0 : BLE_ATT_ERR_INSUFFICIENT_RES;
32
33     case BLE_GATT_ACCESS_OP_WRITE_CHR: {
34         uint16_t len = OS_MBUF_PKTLEN(ctxt->om);          // longueur locale
35         if (len == 0) return 0;
36         if (len >= BLE_MESSAGE_VALUE_MAX) return BLE_ATT_ERR_INVALID_ATTR_VALUE_LEN;
37         if (ble_hs_mbuf_to_flat(ctxt->om, ble_message, len, NULL) != 0)
38             return BLE_ATT_ERR_UNLIKELY;
39         ble_message_len = len;                            // publiée après validation
40         ble_message[len] = '\0';
41         // 'T' (0x54) : accès accordé ; 'F' (0x46) : accès refusé
42         return 0;
43     }
44     default:
45         return BLE_ATT_ERR_UNLIKELY;
46     }
47 }
48
49 static const struct ble_gatt_chr_def gatt_chr_code[] = {
50     { .uuid = BLE_UUID16_DECLARE(GATT_CHR_CODE_UUID),
```

```
      .access_cb = ble_svc_access_cb,
      .flags = BLE_GATT_CHR_F_READ | BLE_GATT_CHR_F_WRITE },
    { 0 }
};

static const struct ble_gatt_svc_def gatt_svcs[] = {
    { .type = BLE_GATT_SVC_TYPE_PRIMARY,
      .uuid = BLE_UUID16_DECLARE(GATT_SVC_UUID),
      .characteristics = gatt_chr_code },
    { 0 }
};

void ble_gatt_svr_init(void) {
    ble_svc_gap_init();
    ble_svc_gatt_init();
    ble_gatts_count_cfg(gatt_svcs);
    ble_gatts_add_svcs(gatt_svcs);
}

// dans app_main()
//   nvs_flash_init() (+ effacement si nécessaire)
//   nimble_port_init();
//   ble_hs_cfg.sync_cb = ble_app_on_sync;
//   ble_gatt_svr_init();
//   nimble_port_freertos_init(ble_host_task);
```

B Commandes utiles

Listing 6 – Compilation, flash et suivi

```
source ~/.espressif/tools/activate_idf_v6.1.sh   # active l'environnement ESP-IDF
idf.py set-target esp32
idf.py menuconfig                                # Component config > Bluetooth > NimBLE
idf.py build
idf.py -p /dev/ttyUSB0 flash monitor
```

C Gestion de version

Le code est versionné sur GitHub. Le dépôt a dû être réinitialisé après qu'un jeton d'accès a été commité par erreur ; GitHub bloque ce type de poussée (*push protection*). Règles retenues : ne jamais écrire de secret dans le code, l'exclure par `.gitignore`, et préférer une clé SSH à un jeton enregistré en clair.

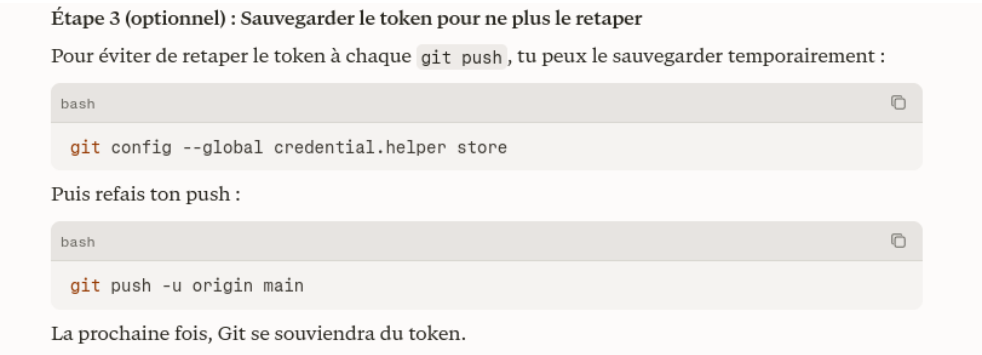


Figure 18 – Mémorisation du jeton avec `credential.helper store` (le jeton est alors enregistré en clair dans le dossier personnel ; une clé SSH est préférable)

Références

[1] Documentation ESP-IDF, pilote I2C : docs.espressif.com, section I2C

[2] Fiche de la carte Freenove ESP32-WROVER : docs.cirkitdesigner.com, Freenove ESP32-WROVER

[3] Documentation ESP-IDF (racine) : `https://docs.espressif.com/projects/esp-idf/`

[4] NXP, *UM10204 — I2C-bus specification and user manual*

[5] Fiches techniques du contrôleur HD44780 (Hitachi) et de l'expandeur PCF8574 (NXP / Texas Instruments)

[6] Documentation Apache Mynewt NimBLE : `https://mynewt.apache.org/`

[7] Bluetooth SIG, spécifications GATT / GAP : bluetooth.com/specifications